

Simplicity Scales

Bebop: A Branchless Data Interchange Format and RPC Protocol

Andrew Sampson

6OVER3 Institute

a@6over3.com

Ronny Chan

6OVER3 Institute

r@6over3.com

Yuta Saito

kateinoigakukun@gmail.com

February 2026

Abstract

The dominant data interchange formats encode integers using a variable number of bytes or represent floating-point numbers as variable-length UTF-8 strings. The decoder must inspect each byte for a continuation bit or parse each character individually, producing data-dependent branches that stall modern CPU pipelines. Protocol Buffers pays this cost on every integer, field tag, and length prefix. JSON pays it on every value.

We present Bebop, a serialization format where every data type uses a fixed number of bytes. A 32-bit integer is always four bytes. Decoding becomes a single memory read with no conditionals. Across 19 decode workloads, Bebop decodes 9–213 $\times$ faster than Protocol Buffers. On a 1536-dimension embedding vector, Bebop decodes in 2.8 nanoseconds versus 111 nanoseconds for Protocol Buffers and 4.69 microseconds for simdjson, a 1,675 $\times$ gap. On records above 64 KB, the decoder achieves 86% of peak memory bandwidth. The CPU is no longer the bottleneck.

We also present a transport-agnostic RPC protocol built on the same wire format. The protocol introduces batch pipelining, where dependent cross-service calls execute in a single round trip with server-side dependency resolution. It deploys over HTTP/1.1, HTTP/2, and binary transports without proxies, removing the HTTP/2 requirement that limits gRPC on serverless platforms and in browsers.

1 Introduction

Transport and storage bandwidth are scaling faster than the compute that sits between them. PCIe x16 throughput grew from 16 GB/s in 2010 to 121 GB/s in 2022 [13]. IEEE 802.3bs ratified 400 Gb/s Ethernet in 2017 [14]. Single-mode fiber has carried 402 Tb/s over 50 km in laboratory demonstration [16]. CXL 3.0 extends cache-coherent memory access across chassis boundaries [15]. Flash read bandwidth scales by ganging devices in parallel, and John Carmack has observed that pipelined flash arrays should already be viable for inference serving if accelerator vendors agreed on a high-speed interface [17]. These are different technologies on the same trajectory. Moving data is getting cheaper.

Processor single-thread performance has not kept pace. Hennessy and Patterson measured the decline in their 2018 Turing Lecture [11]. Annual improvement ran at 52% during the RISC era, dropped to 22% after Dennard scaling broke down at the 90 nm node around 2004 [12], and fell to roughly 3% by 2015. The gap between bandwidth growth and compute growth widens each generation.

Serialization formats carry assumptions about which resource is scarce. Protocol Buffers shipped in 2008 [3] when 1 GbE was standard data center networking. Varint encoding trades CPU cycles for fewer bytes on the wire, a reasonable choice when bandwidth cost dominated. JSON requires character-by-character parsing but provides human readability, a tradeoff that made sense when most payloads were small API responses. Kenton Varda, who maintained Protocol Buffers at Google, noted that the original inventor’s own notes described varint as “a poorly-chosen format due to excessive branching” [2]. His later project Cap’n Proto [4] uses fixed-width encoding to avoid this overhead. The CPU cost was recognized early. The bandwidth context has changed since.

Bebop uses fixed-width encoding for every data type. A 32-bit integer is always 4 bytes. Decoding reduces to a single memory read with no conditionals. In benchmarks across 19 decode workloads, Bebop decodes 9–213× faster than Protocol Buffers. On a 1536-dimension embedding vector, Bebop decodes in 2.8 nanoseconds; simdjson [10], the fastest general-purpose JSON parser, takes 4.69 microseconds on equivalent data, a 1,675× gap. The decoder achieves 86% of peak memory bandwidth on records above 64 KB. At that point, the CPU has nothing left to stall on. LinkedIn found that switching from JSON to Protocol Buffers reduced P99 latency by 60% for large payloads [33]; the gap between Protocol Buffers and Bebop is larger still.

Fixed-width encoding costs more bytes for small integers. In Section 4, OrderLarge with arrays of 100 small integers produces 1,240 bytes in Bebop versus 423 in Protocol Buffers. For workloads dominated by floats, embeddings, and timestamps, the wire size penalty is negligible. Compression narrows the gap further. With Brotli, all three formats produce ML payloads within 2% of each other. The decode performance difference persists regardless of compression.

1.1 Contributions

This paper makes the following contributions:

1. A wire format specification optimized for decode throughput rather than wire compactness
2. Empirical comparison against Protocol Buffers, MessagePack, and simdjson across 19 decode and 23 encode workloads representing ML inference, event streaming, and recursive data structures
3. A schema language with compile-time extensibility through embedded Lua scripting
4. Reference implementations in C achieving 86% memory bandwidth utilization during decode
5. An RPC protocol with batch pipelining for dependent cross-service calls, reducing round trips without requiring application-level coordination

2 Design Principles

2.1 Fixed-Width Encoding

Every numeric type in Bebop has a fixed wire size. A `uint32` is always 4 bytes. A `float64` is always 8 bytes. Length prefixes are always 4 bytes.

The decode operation for a 32-bit integer reduces to a single load instruction:

```
// Bebop decode: one load, no branches
uint32_t value = *(uint32_t*)input;
input += 4;
```

In comparison, Protocol Buffers varint decode loops until finding a byte without the continuation bit set:

```
// Protobuf varint decode: branch per byte
uint32_t value = 0, shift = 0;
while (*input & 0x80) {
    value |= (*input++ & 0x7f) << shift;
    shift += 7;
}
value |= *input++ << shift;
```

This varint loop during decode has unpredictable iteration count when integer values vary, causing branch misprediction penalties on modern CPUs.

2.1.1 Expected Encoding Size

For an unsigned 32-bit integer $v > 0$, varint encoding uses $\lceil (\log_2 v + 1) / 7 \rceil$ bytes. The value $v = 0$ uses 1 byte. Fixed-width encoding always uses 4 bytes. This analysis covers only non-negative integers; signed integers have a pathological case where negative values always use maximum bytes (see Section 2.1.3).

For unsigned integers uniformly distributed over $[0, N]$, we can compute the expected varint size by counting how many values fall into each byte-width bucket. Values in $[0, 2^7 - 1]$ use 1 byte, values in $[2^7, 2^{14} - 1]$ use 2 bytes, and so on:

$$E[\text{varint}] = \frac{1}{N+1} \sum_{k=1}^5 k \cdot |\{v \in [0, N] : 2^{7(k-1)} \leq v < 2^{7k}\}| \quad (1)$$

where the $k = 1$ bucket includes $v = 0$.

Figure 1 shows this tradeoff. The left axis shows wire size: varint uses fewer bytes for small values, crossing over at $N > 2^{28}$. The right axis shows decode latency: fixed-width decoding is constant (one load instruction), while varint decoding increases with byte count due to per-byte branching. For ML workloads with large values, fixed-width encoding is both smaller *and* faster.

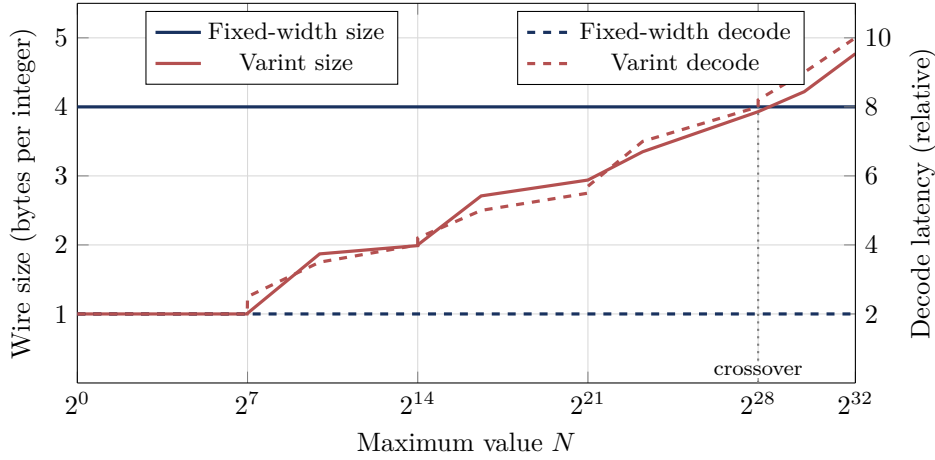


Figure 1: Wire size (solid lines, left axis) vs decode latency (dashed lines, right axis). Varint is smaller for small, positive values but always slower to decode. Above 2^{28} , fixed-width wins on both metrics.

Real-world integer distributions are rarely uniform. Zipfian distributions (common in identifiers, counters, and network data) concentrate probability mass on small values, favoring varint. In contrast, ML workloads, timestamps, and cryptographic hashes have near-uniform distributions over large ranges, where fixed-width encoding wins.

2.1.2 Branch Misprediction Cost

Modern CPUs use speculative execution with branch prediction. A mispredicted branch flushes the pipeline, wasting all in-flight work from fetch through execute. The penalty depends on pipeline depth and microarchitecture; Eyerman et al. report that on deeply pipelined superscalar processors “the misprediction delay is between 10 and 20 clock cycles” [18]. Fog’s measurements on current x86 hardware fall within this range: Skylake at 16–17 cycles, Ice Lake at 17–21 cycles depending on μ op cache residency, and AMD Zen 1–2 at approximately 19 cycles [19]. Apple M1 and Intel Golden Cove show similar penalties [20, 21].

The key difference: varint decoding has data-dependent branches (one per byte), while fixed-width decoding has none. The branch predictor can learn patterns when values are consistent (e.g., always 1–2 bytes), but struggles when byte counts vary. On mixed-size workloads, misprediction adds 4–7 cycles per integer. Fixed-width decode costs 3–4 cycles total (a single load from L1 cache).

The gap widens with value diversity. Workloads mixing small counters with large timestamps see the highest misprediction rates. Section 4 shows measured performance across both cases.

2.1.3 Signed Integer Encoding

Varint has a pathological case: -1 requires 10 bytes because protobuf sign-extends int32 to 64 bits on the wire.

Value	Varint (int32)	Fixed-width
-1	ff ff ff ff ff ff ff ff ff 01 (10 bytes)	ff ff ff ff (4 bytes)
-2	fe ff ff ff ff ff ff ff ff 01 (10 bytes)	fe ff ff ff (4 bytes)

Every negative int32 uses 10 varint bytes. Protocol Buffers addresses this with `sint32` (zigzag encoding) and `fixed32`, but choosing wrong silently inflates wire size.

Bebop uses one encoding per width. `int32` is always 4 bytes regardless of sign.

2.2 Structs vs Messages

Bebop provides two aggregate types with different tradeoffs:

- **Structs:** positional encoding, no tags, no length prefix. Zero overhead. Cannot evolve—any field change is breaking.
- **Messages:** tagged fields (1-byte tags), length-prefixed. 37% overhead on small records, but fields can be added or removed without breaking existing readers.

Protocol Buffers uses tagged encoding everywhere. BeBop lets you choose per-type. Use structs for performance-critical inner types (embeddings, coordinates, points); use messages for top-level API types that may evolve. Section 5.14 details the evolution rules.

Messages also distinguish “not set” from “set to default value.” Proto3 removed this for scalars [5]. BeBop preserves it.

3 Wire Format Specification

All multi-byte integers use little-endian byte order.

3.1 Primitive Types

Type	Size	Encoding
bool	1 byte	0x00 = false, non-zero = true
byte	1 byte	Unsigned 8-bit integer
int8	1 byte	Signed 8-bit, two's complement
int16, uint16	2 bytes	Little-endian
int32, uint32	4 bytes	Little-endian
int64, uint64	8 bytes	Little-endian
float32	4 bytes	IEEE 754 binary32
float64	8 bytes	IEEE 754 binary64

Table 1: Standard primitive types

3.2 Extended Numeric Types

Bebop includes types commonly used in ML workloads:

Type	Size	Description
int128	16 bytes	Signed 128-bit integer. Low 8 bytes first, then high 8 bytes. Used for accumulators and feature hashes.
uint128	16 bytes	Unsigned 128-bit integer. Same encoding as int128.
float16	2 bytes	IEEE 754 binary16 (half precision). 1 sign bit, 5 exponent bits, 10 mantissa bits. Range ± 65504 , precision 3–4 significant digits.
bfloat16	2 bytes	Brain floating point format. 1 sign bit, 8 exponent bits, 7 mantissa bits. Same range as float32, precision 2–3 digits. Common in TPU inference.

Table 2: Extended numeric types for ML workloads

3.3 Temporal Types

3.3.1 timestamp

Absolute point in time: seconds and nanoseconds since Unix epoch (1970-01-01 00:00:00 UTC), with optional timezone offset in signed milliseconds. Total size 16 bytes. Use for event times, creation dates, expiration times, audit logs.

```
e8 03 00 00 00 00 00 00 sec=1000
00 ca 9a 3b ns=999999488
80 62 ee 01 offsetms = 32400000
offset 0: int64, offset 8: int32, offset 12: int32
```

3.3.2 duration

Signed time span: seconds and nanoseconds. Total size 12 bytes. For negative durations, both fields are negative or zero. Use for timeouts, intervals, latency measurements.

```
3c 00 00 00 00 00 00 00 sec = 60
00 00 00 00 ns = 0
offset 0: int64, offset 8: int32
```

3.4 Identifiers

uuid: 16 bytes matching the canonical hex string byte-for-byte.

```
550e8400-e29b-41d4-a716-446655440000:
55 0e 84 00 time_low
e2 9b time_mid
41 d4 time_hi
a7 16 clk_seq
44 66 55 44 00 00 node
```

3.5 Strings

4-byte length prefix (byte count), followed by UTF-8 content, followed by a 1-byte null terminator.

Total wire size: 4 + length + 1 bytes.

The null terminator enables zero-copy access: decoded strings point directly into the input buffer.

```
"hello" encodes as:
05 00 00 00 length = 5
68 65 6c 6c 6f "hello"
00 NUL terminator
```

3.6 Arrays

Dynamic arrays have a 4-byte count prefix followed by elements encoded sequentially.

Fixed arrays (e.g., `byte[4]`) have no prefix; the element count is known at compile time.

Maximum fixed array size is 65535 elements.

```
int32[] = [1, 2, 3]
03 00 00 00  count = 3
01 00 00 00  [0] = 1
02 00 00 00  [1] = 2
03 00 00 00  [2] = 3

byte[4] = [0xDE, 0xAD, 0xBE, 0xEF]
de ad be ef  4 bytes, no prefix
```

3.7 Maps

4-byte count prefix followed by key-value pairs encoded sequentially.

Valid key types: integers, `bool`, `string`, `uuid`.

Floating-point types are not valid map keys due to equality comparison issues with NaN and signed zeros.

```
map[uint8, int32] = {1: 100, 2: 200}
02 00 00 00  count = 2
01  key = 1
64 00 00 00  value = 100
02  key = 2
c8 00 00 00  value = 200
```

3.8 Structs

Fields encode in definition order with no tags and no padding.

Nested structs encode inline. A struct containing another struct has no additional overhead.

Empty structs encode as zero bytes.

```
struct Point { x: float32; y: float32; }
Point { x: 1.0, y: 2.0 }

00 00 80 3f  x = 1.0 (IEEE 754)
00 00 00 40  y = 2.0 (IEEE 754)
```

3.9 Messages

Messages have a 4-byte length prefix, followed by tagged fields, followed by a `0x00` end marker.

Each field is encoded as: 1-byte tag, then the field value.

Absent fields are not encoded. Unknown tags are skipped by decoders. Tags must be in range 1–255.

```
message Request { id(1): int32; name(2):
string; }
Request { id: 42, name: "test" }

10 00 00 00  length = 16 bytes
      01  tag = 1 (id)
2a 00 00 00  value = 42
      02  tag = 2 (name)
04 00 00 00  string length = 4
74 65 73 74  "test"
      00  NUL terminator
      00  end marker
```

3.10 Unions

Unions have a 4-byte length prefix, followed by a 1-byte discriminator, followed by the branch content.

Discriminators must be in range 0–255.

```
union Shape { Circle(1): { radius: float32;
}; }
Shape.Circle { radius: 5.0 }

05 00 00 00  length = 5 bytes
      01  discriminator = 1
00 00 a0 40  radius = 5.0
```

3.11 Complete Example

```
struct Coord {
  x: float32;
  y: float32;
}
message Location {
  name(1): string;
  pos(2): Coord;
  alt(3): float32;
}
```

Location { name: "HQ", pos: {1.0, 2.0}, alt: 100.0 }

```
17 00 00 00  length = 23 bytes
      01  tag 1 (name)
02 00 00 00  string length = 2
48 51 00  "HQ" + null
      02  tag 2 (pos)
00 00 80 3f  pos.x = 1.0
00 00 00 40  pos.y = 2.0
      03  tag 3 (alt)
00 00 c8 42  alt = 100.0
      00  end of message
```

Total: 27 bytes

4 Evaluation

4.1 Experimental Setup

Parameter	Value
Hardware	Apple Mac Studio (M3 Ultra)
CPU cores	28
L1 data cache	64 KB
L2 unified cache	4 MB
Compiler	Clang 17, -O3
CPU scaling	Disabled

We evaluated four systems: Bebop (C runtime), protobuf-c 1.5, msgpack-c 6.1, and simdjson 4.2 for JSON parsing comparison. Each benchmark ran 10 iterations; we report the mean. Across all four systems and 149 total benchmarks, coefficient of variation (CV) averaged 1.65%, indicating low variance and stable measurements. CV was higher on recursive structures (up to 17% on deep trees) where memory access patterns are less predictable.

4.2 Benchmark Workloads

Category	Schema	Description
ML Inference	Embedding768/1536	Single vector, bfloat16
	EmbeddingBatch	32 vectors
	TensorShard	64KB model weight slice
	InferenceResponse	Batch + metadata
LLM Streaming	LLMChunk	Streaming tokens with logprobs
	ChunkedText	Text with span annotations
Event Telemetry	EventSmall	ID, timestamp, payload
	EventLarge	8KB payload variant
API Payloads	PersonSmall/Large	Contact record
	OrderSmall/Large	E-commerce order
	DocumentSmall/Large	Nested document
Recursive	TreeDeep	Binary tree, $d=10$, 1023 nodes
	TreeWide	$b=100$ children, flat
	JsonValue	Union for JSON types

Table 3: 23 benchmark schemas in five categories.

4.3 Decode Performance

Table 4 presents decode latency across the three binary formats. `simdjson` parses JSON text rather than decoding binary, so that comparison appears separately in Table 6. `Bebop` decoded faster than Protocol Buffers on all 19 workloads and faster than `MessagePack` on 18 of 19. `MessagePack` won only on `JsonLarge`.

Workload	Protobuf	MsgPack	Bebop	Speedup
<i>ML Inference</i>				
Embedding768	98.34 ns	62.93 ns	2.91 ns	33.8×
Embedding1536	111.12 ns	63.07 ns	2.80 ns	39.7×
EmbeddingBatch	1.14 μ s	270.96 ns	25.75 ns	44.3×
TensorShardLarge	1.46 μ s	107.90 ns	6.86 ns	212.8×
InferenceResponse	646.69 ns	231.95 ns	17.65 ns	36.6×
<i>LLM Streaming</i>				
LLMChunkLarge	14.72 μ s	5.28 μ s	677.15 ns	21.7×
ChunkedText	50.76 μ s	13.59 μ s	3.16 μ s	16.1×
<i>Event Telemetry</i>				
EventSmall	104.41 ns	84.44 ns	5.94 ns	17.6×
EventLarge	175.20 ns	85.23 ns	6.02 ns	29.1×
<i>API Payloads</i>				
PersonSmall	78.40 ns	72.45 ns	4.10 ns	19.1×
PersonMedium	85.68 ns	78.08 ns	4.21 ns	20.4×
OrderSmall	112.10 ns	112.55 ns	6.14 ns	18.3×
OrderLarge	557.82 ns	757.16 ns	5.85 ns	95.4×
DocumentSmall	71.78 ns	65.54 ns	4.89 ns	14.7×
DocumentLarge	862.73 ns	105.80 ns	52.0 ns	16.6×
<i>Recursive Structures</i>				
TreeDeep	55.00 μ s	23.21 μ s	5.26 μ s	10.5×
TreeWide	4.08 μ s	1.95 μ s	451 ns	9.0×
JsonSmall	521.75 ns	66.16 ns	40.10 ns	13.0×
JsonLarge	13.77 μ s	830.15 ns	1.09 μ s	12.6×

Table 4: Decode latency comparison. Speedup is `Bebop` vs Protocol Buffers.

4.3.1 ML Workload Performance

Figure 2 compares wire encoding for a small embedding (UUID identifier + 4 `bfloat16` values). `Bebop` uses 28 bytes; Protocol Buffers uses 48 bytes. The difference: `Bebop` has a native 16-byte UUID type, while Protocol Buffers encodes UUIDs as 36-byte ASCII strings.

Embedding vectors decode in <3ns with `Bebop` regardless of dimension, compared to 98–111ns with Protocol Buffers (34–40× faster). The `bfloat16` array is a 4-byte count followed by contiguous 16-bit values; decoding is a pointer assignment.

Bebop (28 bytes)	Protocol Buffers (48 bytes)
55 0e 84 00 e2 9b 41 d4 uuid bytes 0–7	0a 24 tag 1, length 36
a7 16 44 66 55 44 00 00 uuid bytes 8–15	35 35 30 65 38 34 ... “550e84...” (ASCII)
04 00 00 00 array length = 4	... 30 30 30 30 36-byte uuid string
80 3f 00 40 40 40 80 40 bfloat16 data	12 08 tag 2, length 8
16B uuid + 4B len + 8B data	80 3f 00 40 40 40 80 40 bfloat16 data
	2B tag + 36B string + 2B tag + 8B data

Figure 2: Wire encoding of a small embedding. Bebop’s native UUID saves 20 bytes versus Protocol Buffers’ string encoding. Hex bytes from actual encoder output.

4.3.2 Recursive Structure Performance

TreeDeep (binary tree, $d=10$, 1023 nodes) decodes in $5.34\mu s$ with Bebop versus $55.00\mu s$ with Protocol Buffers (10× faster). For recursive messages, Bebop’s length prefixes allow skipping subtrees without parsing contents. The speedup comes from predictable memory access, not wire compactness—Protocol Buffers’ varint encoding produces smaller output for trees with small integer values.

4.4 Throughput and Memory Bandwidth

Bebop’s decode performance is bounded by memory bandwidth, not CPU compute. Figure 3 shows bandwidth utilization across record sizes. On cold-cache workloads (data fetched from DRAM), Bebop achieves **86% of peak memory bandwidth** on records above 64KB. This is the meaningful metric for production workloads where data doesn’t fit in cache.

Table 5 shows measured throughput. Values above 819 GB/s (M3 Ultra memory bandwidth [22]) indicate cache-resident data from benchmark iterations—useful for understanding overhead but not representative of cold-cache production loads.

Workload	Throughput	Cache	Notes
TensorShardLarge	9.58 TB/s	L2	64KB fits in L2
Embedding1536	1.10 TB/s	L2	3KB vector
EmbeddingBatch	964.13 GB/s	L2	Batch of 32
EventLarge	644.35 GB/s	L2/DRAM	4KB payload
Embedding768	534.90 GB/s	L2	1.5KB vector
InferenceResponse	355.51 GB/s	L2	Mixed content
OrderLarge	212.50 GB/s	L2	Nested arrays

Table 5: Bebop decode throughput. Values above 819 GB/s indicate L2-resident data.

Bebop’s decode path does minimal computation: bounds checking, pointer arithmetic, occasional type conversion. Most “decode” operations are pointer assignments.

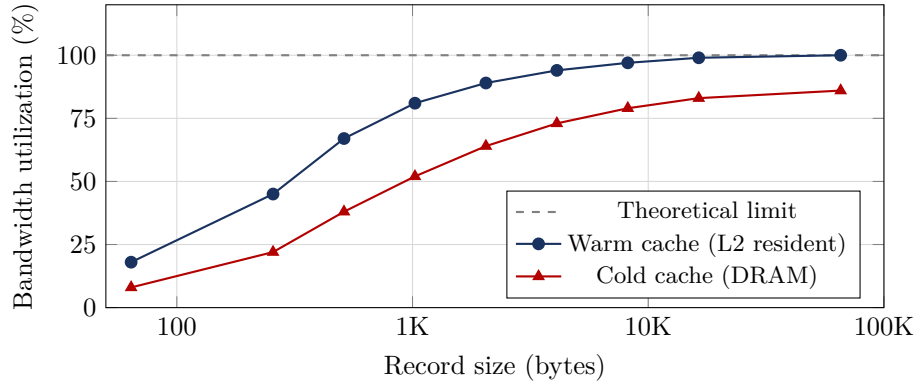


Figure 3: Bandwidth utilization vs record size. Larger records amortize per-record overhead (function call, bounds check, struct initialization) and approach the memory bandwidth limit.

The gap at small record sizes reflects fixed per-record overhead: records under 256 bytes spend more time in function prologues and bounds checks than in actual data movement. Above 4KB, records achieve over 90% bandwidth utilization when cache-resident.

4.4.1 Alignment and Single Instruction, Multiple Data (SIMD)

Structs encode in definition order with no padding (Section 3.8), but in-memory layout differs. The C code generator reorders fields by alignment to minimize padding:

<pre> struct Sensor { bool active; float64 reading; uint16 id; uint32 seq; } </pre> <i>Schema order</i>	→	<pre> typedef struct { double reading; // 8-byte align uint32_t seq; // 4-byte align uint16_t id; // 2-byte align bool active; // 1-byte align } Sensor; </pre> <i>Generated C (sorted by alignment)</i>
--	---	--

Schema authors write fields in logical order; generated code handles layout.

The runtime arena aligns all allocations to `max_align_t`, the strictest fundamental alignment guaranteed by the platform (16 bytes on most 64-bit systems). Decoded structs receive proper alignment regardless of their position in the wire stream. For GPU and TPU transfers requiring even stricter alignment (32, 64, or 128 bytes), the arena accepts a custom allocator. Embedding vectors and tensor data in fixed arrays decode to contiguous memory, suitable for DMA after aligning the containing buffer.

The reference runtime does not use SIMD intrinsics. For `bfloat16[]` arrays, decode is a pointer assignment into arena memory; SIMD would add no benefit. For arrays of small structs, SIMD could parallelize bounds checking, but the current implementation favors portability over architecture-specific optimization.

4.5 Encode Performance

Encode speedups are smaller than decode speedups. This is expected: encoding requires traversing data structures and computing lengths, which involve allocation and branching regardless of wire format. Bebop’s fixed-width encoding eliminates branches during decode, but encoding still requires the same traversal as other formats.

Bebop beat Protocol Buffers on all 22 comparable workloads (1.4–12.6 $\times$) and MessagePack on 15 of 19 (1.2–19.4 $\times$). MessagePack was faster on JsonSmall, JsonLarge, ChunkedText, and DocumentLarge.

Figure 4 compares encode and decode latency across all three binary formats for representative workloads.

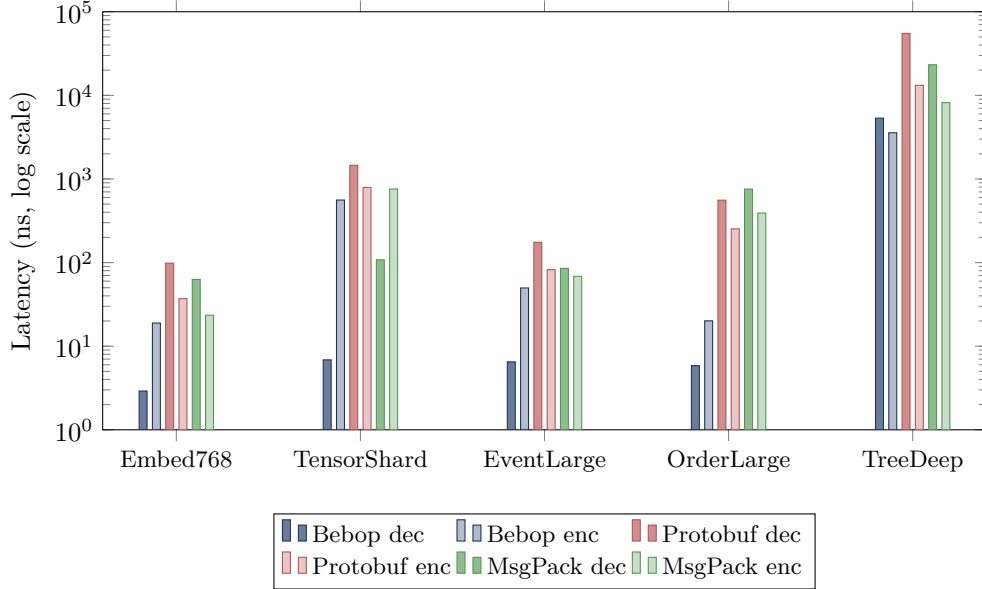


Figure 4: Encode vs decode latency across binary formats. Bebop’s decode advantage (dark blue) is larger than its encode advantage (light blue) because encoding requires data traversal regardless of wire format.

4.6 Comparison with JSON Parsing

This comparison is not apples-to-apples: simdjson parses human-readable text, while Bebop decodes pre-serialized binary. But for systems currently using JSON, this shows the performance cost of that choice.

We compared against simdjson [10], the fastest general-purpose JSON parser, which uses SIMD to accelerate tokenization (2–6 GB/s on typical workloads). On equivalent data, Bebop decode was faster on 17 of 19 workloads (1.2–5741 $\times$). The largest gaps occurred on numeric arrays: parsing “[1.5, 2.5, ...]” as JSON requires character-by-character float

conversion, while Bebop reads IEEE 754 values directly. Simdjson was faster on JsonLarge ($4.2\times$) and JsonSmall ($1.7\times$), where JSON’s native format requires no conversion.

Workload	simdjson	Bebop	Speedup
TensorShardLarge	39.38 μ s	6.86 ns	5741 $\times$
Embedding1536	4.69 μ s	2.80 ns	1675 $\times$
EmbeddingBatch	27.78 μ s	25.75 ns	1079 $\times$
Embedding768	2.26 μ s	2.91 ns	776 $\times$
InferenceResponse	3.93 μ s	17.65 ns	223 $\times$
OrderLarge	283.24 ns	5.85 ns	48 $\times$
DocumentLarge	245.53 ns	52.0 ns	4.7 $\times$
LLMChunkLarge	1.69 μ s	677.15 ns	2.5 $\times$
TreeDeep	6.54 μ s	5.26 μ s	1.2 $\times$

Table 6: simdjson parse vs Bebop decode on equivalent data.

4.7 Roundtrip Latency

Table 7 shows encode-then-decode latency for representative workloads. Speedups range from $4.4\times$ to $35\times$ over Protocol Buffers.

Workload	Protobuf	MsgPack	Bebop	Speedup
PersonSmall	124.40 ns	87.30 ns	11.8 ns	10.5 $\times$
OrderLarge	834.68 ns	1.01 μ s	23.86 ns	35.0 $\times$
EventLarge	238.04 ns	150.00 ns	52.8 ns	4.5 $\times$
TreeDeep	68.93 μ s	30.50 μ s	8.86 μ s	7.8 $\times$

Table 7: Roundtrip latency (encode + decode). Speedup is Bebop vs Protobuf.

4.8 Wire Size

The results confirm the predictions from Equation 1. OrderLarge contains arrays of 100 small integers; varint encodes these in 1–2 bytes each while fixed-width requires 4–8 bytes. MessagePack falls between the two: its schema-less design adds field name overhead, but it uses compact integer encoding.

Compression narrows the gap on most workloads. On ML payloads (embedding vectors, tensor shards), all three formats produce nearly identical compressed sizes—within 2%. The bfloat16 data dominates; framing overhead becomes negligible after compression.

On API payloads with small integers (OrderLarge), compressed Protocol Buffers remains smallest (376 bytes) due to varint’s compact representation of small values. MessagePack compresses to 490 bytes; Bebop to 514 bytes.

Workload	Protobuf	MsgPack	Bebop	PB+brotli	MP+brotli	Bebop+brotli
<i>API payloads (small integer fields)</i>						
PersonSmall	19	35	28	—	—	—
PersonMedium	124	141	130	101	107	101
OrderSmall	34	94	76	—	93	62
OrderLarge	423	664	1,240	376	490	514
<i>Event payloads (byte arrays)</i>						
EventSmall	29	62	42	—	—	—
EventLarge	4,158	4,193	4,170	—	—	—
<i>ML inference (bfloat16 arrays)</i>						
Embedding768	1,573	1,568	1,556	1,193	1,182	1,167
Embedding1536	3,109	3,104	3,092	2,214	2,246	2,219
TensorShardSmall	2,108	2,154	2,132	1,576	1,602	1,574
TensorShardLarge	65,605	65,652	65,624	44,720	44,747	44,711

Table 8: Wire size in bytes. — indicates compression increased size.

EventLarge is dominated by a 4KB random byte payload; no format benefits from compression.

With compression enabled, wire size differences between formats largely disappear. The decode performance gap does not.

5 Schema Language

5.1 Source Encoding

Schema files (.bop) must be valid UTF-8. The compiler rejects files with invalid byte sequences. String literals must also be valid UTF-8; invalid sequences produce compile-time errors.

5.2 File Structure

A schema file has three sections in order:

1. **Header** – edition and package (both optional)
2. **Imports** – import statements
3. **Definitions** – types, constants, services

The `edition` declaration specifies schema language version. The `package` declaration provides a namespace for all definitions.

```
edition = "2026"
package my.app

import "bebop/decorators.bop"
import "shared/types.bop"

struct Point {
  x: float32;
  y: float32;
}
```

5.3 Comments

Three comment styles are supported.

Line comments (`//`) and block comments (`/* */`) are discarded during parsing.

Documentation comments (`///`) on the line immediately before a definition are captured as metadata and appear in generated code.

```
// Line comment

/* Block comment
   spans lines */

/// Documentation comment
/// for the struct below
struct User {
  name: string;
}
```

5.4 Literals and Escape Sequences

String literals use double quotes (`"`) or single quotes (`'`). Literal newlines are allowed within strings.

Escape	Output	Notes
<code>\\</code>	Backslash	
<code>\n</code>	Newline (LF)	
<code>\r</code>	Carriage return	
<code>\t</code>	Tab	
<code>\0</code>	Null byte	
<code>\"</code>	Double quote	Also: <code>"</code> inside double-quoted strings
<code>\'</code>	Single quote	Also: <code>'</code> inside single-quoted strings
<code>\u{XXXX}</code>	Unicode codepoint	1–6 hex digits, produces UTF-8

Numeric literals: decimal, hexadecimal (`0xFF`), scientific (`1.23e10`). Special floats: `inf`, `-inf`, `nan`. Byte arrays use a `b` prefix: `b"\x89PNG"`.

Timestamps use ISO 8601 (`"2024-01-15T10:30:00Z"`). Timezone offsets and nanosecond precision are supported. The timezone offset can be expressed up to millisecond precision in ISO 8601-2:2019 format (`"2024-01-15T10:30:00+12:00:01.133"`).

Durations use suffixes: `"1h30m"`, `"500ms"`, `"10us"`.

String constants support environment variable substitution: `"$(VAR)"` resolves at compile time.

5.5 Type Reference

See Tables 1 and 2 in Section 3 for wire encoding details. Type aliases: `uint8` = `byte`, `half` = `float16`, `bf16` = `bfloat16`, `guid` = `uuid`.

5.6 Enumerations

Every enum must have a member with value 0 (the default).

Base type defaults to `uint32`. Override with a colon to specify a different underlying type.

```
enum Status : uint8 {
    UNKNOWN = 0;
    ACTIVE = 1;
    SUSPENDED = 2;
}
```

5.7 Structs

By default, structs are immutable in generated code. Use `mut` for mutable structs. See Section 2.5 for wire encoding.

```
struct Color {  
  r: byte;  
  g: byte;  
  b: byte;  
  a: byte;  
}  
  
mut struct MutablePoint {  
  x: float32;  
  y: float32;  
}
```

5.8 Messages

Tags must be unique integers 1–255. Don't reuse old tags with different types. See Sections 2.5 and 2.8 for wire encoding and evolution rules.

```
message UserProfile {  
  id(1): uuid;  
  name(2): string;  
  email(3): string;  
  created(4): timestamp;  
}
```

5.9 Unions

Discriminated variant. Branches can be inline structs, inline messages, or references to existing types.

```
union Result {  
  Success(1): {  
    value: string;  
  };  
  Error(2): {  
    code: int32;  
    message: string;  
  };  
}
```

5.10 Services

Services define RPC interfaces. The `stream` keyword indicates multiple messages. Request and response types must be named struct, message, or union definitions. Primitives and inline types are not allowed. Services can include methods from other services using the `with` keyword for composition.

```
service BaseService {
    GetStatus(Req): StatusRes;
}

service ChatService with BaseService {
    Send(Message): Ack;
    Subscribe(Req): stream Event;
    Upload(stream Chunk): Summary;
    Chat(stream Msg): stream Msg;
}
```

Type	Syntax	Description
Unary	Method(Req): Res	Single request, single response
Server stream	Method(Req): stream Res	Request, multiple responses
Client stream	Method(stream Req): Res	Multiple requests, response
Bidirectional	Method(stream): stream	Streams both directions

5.11 Constants

Named compile-time values accessible in generated code. Timestamp literals use ISO 8601 format. Duration literals use time unit suffixes: h, m, s, ms, us, ns. Byte arrays use a `b` prefix with escape sequences.

```
const int32 MAX_SIZE = 1024;
const string HOST = "localhost";
const timestamp EPOCH =
    "1970-01-01T00:00:00Z";
const duration TIMEOUT = "30s";
const byte[] PNG_MAGIC =
    b"\x89PNG\r\n\x1a\n";
```

5.12 Visibility

Top-level definitions are exported by default. Use `local` to make them file-private. Nested definitions (types inside structs, messages, or unions) are local by default. Use `export` to make them accessible.

```
struct PublicType {}
local struct PrivateType {}

struct Outer {
    struct LocalInner {}
    export struct PublicInner {}
}
```

5.13 Compile-Time Extensibility

Decorators provide schema annotations processed at compile time. Code generators receive decorator arguments through the standard plugin protocol. The `export` block extends this by computing derived values from the full schema context.

Decorator syntax:

- `targets` – where decorator applies
- `param name!: Type` – required
- `param name?: Type` – optional
- `validate [[ lua ]]` – reject invalid usage
- `export [[ lua ]]` – produce plugin metadata

Valid targets: `ENUM`, `STRUCT`, `MESSAGE`, `UNION`, `FIELD`, `SERVICE`, `METHOD`, `BRANCH`, `ALL`.

The `validate` block receives parameters plus a target table (`kind`, `name`, `parent`).

```
#decorator(range) {
  targets = FIELD
  param min!: int32
  param max!: int32
  validate [[
    if min >= max then
      error("min must be < max", self.min.
        span)
    end
  ]]
  export [[
    return { range_min = min, range_max =
      max,
      width = max - min }
  ]]
}
```

Export blocks compute derived data using the full schema context. Plugins already receive raw decorator arguments through the descriptor. The export block produces values that require compile-time computation or access to the target element's metadata.

```
#decorator(indexed) {
  targets = FIELD
  param unique?: bool
  export [[
    local t, f = target.parent, target.name
    return {
      index_name = t.."..".."f.."idx",
      table_name = t, column_name = f,
      is_unique = unique or false
    }
  ]]
}
```

A database code generator receives the computed `index_name` directly (e.g., `"User_email_idx"`).

Export blocks can aggregate data across the schema for deprecation reports, validation constraints, or reflection tables.

5.14 Schema Evolution

Type	Change	Safe?	Notes
Message	Add field	✓	Use new tag; old readers ignore
	Deprecate field	✓	Skipped on wire; don't reuse tag
	Rename field	✓	Names not on wire
	Change field type	✗	Never reuse tag with different type
	Change tag number	✗	Equivalent to remove + add
Struct	Add field	✗	Positional encoding; no tags
	Remove field	✗	Create versioned type instead
	Reorder fields	✗	Or convert to message
	Change field type	✗	
Union	Add branch	✓	
	Remove branch	✗	Decode fails for existing data
	Change branch type	✗	
Enum	Add value	✓	
	Remove value	✗	Existing data may contain it
	Change base type	✗	e.g., uint8 to uint32

Table 9: Schema evolution compatibility. ✓ = backward compatible, ✗ = breaking change.

6 Implementation

6.1 Compiler

The Bebop compiler (`bebopc`) is written in portable C with no external dependencies. The resulting binary is under 2MB and runs on Linux, macOS, Windows, and BSD.

```
$ bebopc build schema.bop --c_out=./generated
$ bebopc build schema.bop --typescript_out=./ts
```

6.2 Plugin Architecture

Code generators are standalone executables named `bebopc-gen-$NAME`. The compiler discovers them in `PATH` and invokes them via `--$NAME_out=DIR`.

Communication uses Bebop-encoded messages on stdin/stdout. Plugins can be written in any language with a Bebop runtime.

Request fields:

- `files_to_generate` – source files from command line
- `parameter` – from `--$NAME_opt= ...`
- `schemas` – descriptors, topologically sorted

Response fields:

- `error` – fatal error message (if any)
- `files` – generated file name + content pairs
- `diagnostics` – warnings/errors with source spans

Plugins can extend files from other plugins using insertion points, markers that later plugins can target.

```
message CodeGeneratorRequest {
  files_to_generate(1): string[];
  parameter(2): string;
  compiler_version(3): Version;
  schemas(4): SchemaDescriptor[];
}

message CodeGeneratorResponse {
  error(1): string;
  files(2): GeneratedFile[];
  diagnostics(3): Diagnostic[];
}
```

6.3 Descriptor Format

The compiled schema representation uses Bebop's own wire format. Descriptors are passed to plugins and can be used for runtime reflection.

Structure:

- `DescriptorSet` – root container
- `SchemaDescriptor[]` – one per .bop file
- `DefinitionDescriptor[]` – types, services, constants

Definitions are topologically sorted: dependencies appear before dependents. Process sequentially for single-pass code generation.

Each definition includes its kind (enum, struct, message, union, service, const), fully-qualified name, documentation from `///` comments, visibility, and applied decorators with their exported data.

```
message DefinitionDescriptor {
  kind(1): DefinitionKind;
  name(2): string;
  fqcn(3): string;
  documentation(4): string;
  visibility(5): Visibility;
  decorators(6): DecoratorUsage
    [];
  nested(7): DefinitionDescriptor
    [];
  // kind-specific body:
  enum_def(8): EnumDef;
  struct_def(9): StructDef;
  message_def(10): MessageDef;
  union_def(11): UnionDef;
  service_def(12): ServiceDef;
  const_def(13): ConstDef;
}
```

Type references use `TypeDescriptor` with a `kind` field (`BOOL`, `INT32`, `STRING`, `ARRAY`, `MAP`, `DEFINED`, etc.) and recursive structure for nested types. Service methods include a stable 32-bit routing ID computed from `/ServiceName/MethodName` using `MurmurHash3` with the `lowbias32` finalizer [34] (bias 0.17 vs. standard 0.23).

7 RPC Protocol

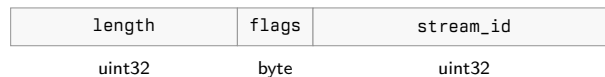
7.1 Motivation

In microservice architectures, serialization sits inside the RPC layer. The framing, error encoding, metadata handling, and call multiplexing all add latency on top of the serialization itself. gRPC couples Protocol Buffers to HTTP/2 with its own framing protocol, length-prefixed messages, and HPACK-compressed headers [23]. Replacing the serialization format means replacing the RPC stack.

Bebop's RPC protocol uses Bebop encoding for every layer: frame headers, call headers, error payloads, metadata, the batch protocol, and service discovery responses. An implementation that can decode Bebop messages can decode every part of the protocol. One encoding, one set of generated types, one decoder path.

7.2 Protocol Design

A Bebop RPC frame has a fixed 9-byte header:



The `length` field is the payload byte count. `flags` is a bitfield combining `END_STREAM`, `ERROR`, `COMPRESSED`, `TRAILER`, and `CURSOR`. `stream_id` provides multiplexing on transports that require it. A complete unary RPC uses 18 bytes of framing overhead: 9 bytes in each direction.

The protocol supports four method types: unary (single request, single response), server streaming (one request, multiple responses), client streaming (multiple requests, one response), and duplex streaming (both directions). These match gRPC's taxonomy, and status codes 0–16 align with gRPC's definitions, so bridging between the two protocols requires no code remapping.

Method dispatch uses a 4-byte hash of `/ServiceName/MethodName` computed at schema compile time via MurmurHash3. The router performs integer comparison instead of string matching on every incoming call.

The protocol is transport-agnostic. On HTTP/1.1 and HTTP/2, each request-response pair maps to a standard HTTP exchange, with metadata carried in HTTP headers. On binary transports (TCP, WebSocket, IPC, Unix sockets), the full frame protocol runs directly, with a `CallHeader` initiating each call and stream IDs providing multiplexing. This separation means the same service definition generates handlers that deploy identically on an HTTP load balancer, a WebSocket gateway, or a raw TCP socket.

7.3 Batch Pipelining

Dependent cross-service calls typically require sequential round trips. Fetching a user record and then querying their friends list costs two round trips: the second call cannot begin until the first completes. In a chain of N dependent calls, latency grows as $N \times \text{RTT}$.

Bebop RPC provides a batch protocol that collapses dependent calls into a single round trip. Each call in a batch carries an `input_from` field. When set to -1 , the call uses its own payload. When set to the index of a previous call, the server forwards that call's result as input. The server builds a dependency graph from these references, partitions calls into execution layers, and runs all calls within a layer concurrently. Layer $k + 1$ waits only for the calls in layer k that it depends on.

```
message BatchCall {
  call_id(1): int32;
  method_id(2): uint32;
  payload(3): byte[];
  input_from(4): int32; // -1 = use payload, >=0 = forward result
}
```

If a call fails, all calls that depend on it also fail with status `INVALID_ARGUMENT`. If the batch deadline expires mid-execution, remaining calls fail with `DEADLINE_EXCEEDED`. Server-stream methods within a batch buffer their results into arrays; true streaming semantics would require multiplexed response framing, so client-stream and duplex methods are excluded from batching.

gRPC has no built-in batching primitive [24]. Dependent calls require sequential round trips or application-level batch message types defined in the Protobuf schema. Bebop's batch protocol handles dependency resolution and concurrent execution at the framework level.

7.4 Deadline Propagation

Bebop RPC transmits deadlines as absolute timestamps with nanosecond precision. Every downstream hop checks the same wall-clock cutoff. The Google SRE book identifies deadline propagation as a defense against cascading failures [32]. Without propagation, downstream services continue working on requests the caller has already abandoned.

gRPC converts absolute deadlines to relative timeouts with elapsed time deducted at each hop [25]. Bebop avoids the deduction step by transmitting the absolute timestamp directly on binary transports. On HTTP transports, the deadline is a millisecond Unix timestamp in the `bebop-deadline` header. Both representations refer to the same wall-clock instant, so no rounding accumulates across hops.

7.5 Stream Cursors

A server-stream call that delivers 10,000 results and drops at result 9,500 has two options without protocol support: re-request the entire stream, or track progress in the application layer. gRPC takes the second approach—the protocol has no resumption mechanism, so handlers implement their own checkpointing.

Bebop RPC provides cursor-based resumption at the frame level. The `CallHeader` carries a `cursor` field (uint64). On the first call this is zero. On reconnection, the client sends the last cursor it fully processed. The handler reads `RpcContext.cursor` and skips past already-delivered data. In the other direction, response frames carry position markers: when the `CURSOR` flag (0x10) is set, 8 bytes of little-endian uint64 follow the payload. The `length` field counts only payload bytes; the cursor is appended outside it. Not every frame needs a cursor, and a stream may freely mix cursored and non-cursored frames.

What a cursor value means is handler-specific: a database offset, a sequence number, a timestamp, a log position. The protocol treats it as an opaque uint64. Resume logic stays in the call header and wire frames, never in application message types. This separation is also why only unary methods can be dispatched as futures (Section 7.6)—server-stream methods already have a protocol-level mechanism for surviving disconnections.

7.6 Futures

Long-running operations create a tension in synchronous RPC. An ML inference call that takes 30 seconds holds a connection and a server thread for the duration. The caller cannot disconnect and reconnect without losing the result. gRPC’s standard pattern for this is Google’s `google.longrunning.Operations` service [30], which returns an operation ID and requires the client to poll a separate `GetOperation` endpoint until completion.

Bebop RPC replaces polling with push-based delivery using three reserved method IDs:

ID	Method	Type	Payload
2	Dispatch	unary	<code>FutureDispatchRequest</code> → <code>FutureHandle</code>
3	Resolve	server-stream	<code>FutureResolveRequest</code> → <code>FutureResult</code>
4	Cancel	unary	<code>FutureCancelRequest</code> → <code>Empty</code>

A `FutureDispatchRequest` wraps a unary call or batch for background execution. The server registers the work, spawns a task, and returns a `FutureHandle` containing a server-generated v4 UUID. The dispatch call completes in the time it takes to validate the request and allocate the ID; the `deadline` field in the request applies to the inner call’s execution, not to the dispatch itself.

The resolve stream (method 3) is a server-stream connection that pushes `FutureResult` messages as futures complete. Each result contains the future’s UUID and a terminal outcome: either a success payload with response metadata, or an error with a status code.

The `ids` field in the request filters delivery to specific futures; omitting it subscribes to all futures owned by the caller. When specific IDs are requested and some have already completed, the server sends those results immediately before continuing with remaining completions. The inner handler is unaware it is running as a future—the server invokes it identically to a synchronous unary call.

7.6.1 Idempotency and ownership

The `FutureDispatchRequest` carries an optional `idempotency_key` field, a client-generated UUID. If a pending or completed future with the same key exists for the same caller, the server returns the existing handle without dispatching again. Cancellation releases the key so a subsequent dispatch creates a new future. Keys are scoped per caller; two different callers can use the same key without collision.

Every future is bound to a caller identity resolved from peer information: authenticated identity if available, otherwise the connection’s remote address. The server checks this identity on every resolve and cancel operation. A caller that does not own a future receives `PERMISSION_DENIED`.

7.6.2 Retention and storage

The server configures a default retention policy—typically eviction-by-count—that applies to all completed futures. Clients can override this per-dispatch by setting `discard_result` in the dispatch request. When set, the server delivers the result to active resolve streams and immediately discards it—the future is not promised. The handle is still returned so the client can cancel in-flight work, and idempotency keys still deduplicate retries, but rehydration from a saved UUID returns nothing because the result no longer exists. Clients cannot force retention beyond the server’s policy; they can only opt out of it.

The reference implementation defines an asynchronous storage protocol so that the in-memory store can be replaced with database, disk, or tiered-cache backends. Protocol operations are split for composability: persisting a completed result and notifying active subscribers are separate interface methods, allowing a database backend to commit before fanning out to in-memory streams.

7.7 Deployment Model

gRPC requires HTTP/2 [23]. This creates friction in several environments. AWS Lambda’s API Gateway communicates with functions over HTTP/1.1; a Coinbase engineering investigation found gRPC “very close to possible” for unary calls but hit blocking issues with trailing headers, and streaming was unsupported entirely [26]. Cloudflare Workers lack HTTP/2 streaming support in the `workerd` runtime [27]. In browsers, gRPC-Web requires

an Envoy proxy because browser APIs do not expose HTTP/2 framing or trailing headers [28].

Twitch built Twirp [29] as an HTTP/1.1-compatible alternative to gRPC after grpc-go’s embedded HTTP/2 implementation caused production outages. Twirp drops streaming entirely in exchange for deployment simplicity.

Bebop RPC is transport-agnostic. Unary and streaming methods work over HTTP/1.1 through HTTP/3, binary TCP, WebSocket, WebRTC, WebTransport, and IPC. In browsers, any available transport works directly with no proxy. Metadata maps to HTTP headers. Errors map to HTTP status codes. The same `.bop` service definition generates server handlers and typed client stubs for every target language.

7.8 Comparison with gRPC

Feature	gRPC	Bebop RPC
Wire encoding	Protobuf + HTTP/2 framing	Bebop throughout
Frame overhead	5B length prefix + HTTP/2 frame	9B fixed header
Batch pipelining	Application-level	Native with dependency graph
Async dispatch	Polling (<code>google.longrunning</code>)	Push-based futures
Transport	HTTP/2 required	Any
Serverless	Limited	HTTP/1.1 compatible
Method dispatch	String path matching	4-byte hash comparison
Deadline model	Relative timeout with deduction	Absolute timestamp
Status codes	0–16	0–16 aligned + 17–255 app-defined
Browser support	Requires Envoy proxy	Native

Table 10: Feature comparison between gRPC and Bebop RPC.

8 Related Work

Protocol Buffers [1] prioritizes wire compactness through varint encoding. The branching cost during decode was recognized early [2]. Bebop prioritizes decode speed instead.

Cap’n Proto [4] and FlatBuffers [6] optimize for random field access through pointer-based layouts. Bebop targets sequential decode, which matches how ML pipelines consume embedding vectors.

Simple Binary Encoding (SBE) [7] also uses fixed-width encoding, targeting financial systems where latency matters. SBE focuses on FIX protocol semantics and requires fields in schema-defined order. Bebop takes a similar approach to wire encoding but provides a more general-purpose schema language with tagged messages for evolution.

MessagePack [8] embeds type tags in the wire format; Avro [9] includes the schema with each message. Both add per-message overhead that Bebop avoids through code generation.

simdjson [10] uses SIMD to accelerate JSON parsing. Section 4 compares performance.

Bebop’s RPC protocol builds on ideas from several frameworks. gRPC [23] provides RPC over HTTP/2 with Protocol Buffers as the default wire format. Bebop aligns its status codes with gRPC’s and adopts the same four method types, but removes the HTTP/2 requirement and adds batch pipelining. Section 7 details these differences.

Apache Thrift [31] combines an IDL, multiple serialization formats, and transport abstraction in a single framework. Thrift predates gRPC and influenced its design.

Twirp [29] dropped streaming entirely in favor of HTTP/1.1 compatibility and deployment simplicity. Bebop RPC retains streaming by supporting multiple transports.

Google’s long-running operations pattern [30] adds async dispatch to gRPC through a polling-based `Operations` service. The client calls `GetOperation` until the `done` field is true. Bebop’s futures use a push-based resolve stream instead, avoiding repeated round trips but requiring the client to maintain a stream connection.

9 Conclusion

Varint encoding saved bytes at the cost of branch-heavy decode loops. As the gap between bandwidth growth and compute growth widens, that tradeoff ages poorly.

Bebop decodes tensor data $213\times$ faster than Protocol Buffers. Fixed-width encoding eliminates the branch-per-byte loop that varints require. No tag dispatch for structs. No runtime length computation. The decoder reduces to pointer arithmetic and bounds checks, simple enough that it saturates memory bandwidth on large records.

The tradeoff is wire size. Fixed-width integers cost more bytes than varints for small values. Section 4 shows where this matters and where compression eliminates the difference.

Faster serialization matters more when the RPC layer preserves the gain. Batch pipelining turns N dependent calls into a single exchange, futures replace polling for long-running operations with push-based delivery, and HTTP/1.1 compatibility means the protocol deploys where gRPC cannot.

What surprised us was how much performance came from doing less. Fixed sizes, predictable layouts, no per-value decisions. The same choices that make Bebop fast also keep the compiler small. The implementation is 35,000 lines of C with single-pass code generation. Simplicity compounded.

Future work includes SIMD-accelerated array decoding, GPU-side deserialization for direct device memory placement, streaming decode for records larger than available memory, and RPC runtime implementations for additional target languages. The wire format and RPC protocol are stable; these are implementation improvements that maintain compatibility.

Acknowledgments

We thank Tristram Jenkins, PhD, of Tokyo University and Yoonseo Kang for detailed feedback on earlier drafts of this paper.

Availability

Bebop is open source. The compiler, runtime libraries, and benchmark code are available at <https://github.com/6over3>.

References

- [1] Google. *Protocol Buffers*. <https://protobuf.dev/>, 2008.
- [2] K. Varda. Comment on Hacker News, May 2016. <https://news.ycombinator.com/item?id=11657767>
- [3] K. Varda. “Protocol Buffers: Google’s Data Interchange Format.” *Google Open Source Blog*, July 2008. <https://opensource.googleblog.com/2008/07/protocol-buffers-google-data.html>
- [4] K. Varda. *Cap’n Proto*. <https://capnproto.org/>, 2013.
- [5] Google. “Field Presence.” *Protocol Buffers Documentation*, 2020. https://protobuf.dev/programming-guides/field_presence/
- [6] Google. *FlatBuffers*. <https://flatbuffers.dev/>, 2014.
- [7] Real Logic. *Simple Binary Encoding*. <https://github.com/aeron-io/simple-binary-encoding>, 2013.
- [8] S. Furuhashi. *MessagePack*. <https://msgpack.org/>, 2008.
- [9] Apache Software Foundation. *Apache Avro*. <https://avro.apache.org/>, 2009.
- [10] G. Langdale and D. Lemire. “Parsing Gigabytes of JSON per Second.” *The VLDB Journal*, 28(6):941–960, 2019.
- [11] J. L. Hennessy and D. A. Patterson. “A New Golden Age for Computer Architecture.” *Communications of the ACM*, 62(2):48–60, 2019. Based on Turing Lecture delivered at ISCA, June 2018.
- [12] M. Bohr. “A 30 Year Retrospective on Dennard’s MOSFET Scaling Paper.” *IEEE Solid-State Circuits Newsletter*, 12(1):11–13, 2007.
- [13] PCI-SIG. *PCI Express Base Specification Revision 6.0*, Version 1.0, January 2022.
- [14] IEEE. “IEEE Standard for Ethernet – Amendment 10: Media Access Control Parameters, Physical Layers, and Management Parameters for 200 Gb/s and 400 Gb/s Operation.” IEEE Std 802.3bs-2017, December 2017.
- [15] D. Das Sharma, R. Blankenship, and D. Berger. “An Introduction to the Compute Express Link (CXL) Interconnect.” *ACM Computing Surveys*, 56(11):1–37, 2024.
- [16] B. J. Puttnam, H. Furukawa, et al. “402 Tb/s GMI Data-Rate OESCLU-Band Transmission.” Post-deadline paper Th4A.3, Optical Fiber Communication Conference (OFC), San Diego, March 2024.
- [17] J. Carmack. Post on fiber optic delay-line memory and flash bandwidth for AI inference. X (formerly Twitter), February 2026. https://x.com/ID_AA_Carmack/status/2019839335382790342

- [18] S. Eyerman, J. E. Smith, and L. Eeckhout. “Characterizing the Branch Misprediction Penalty.” *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 48–58, 2006.
- [19] A. Fog. “The microarchitecture of Intel, AMD and VIA CPUs: An optimization guide for assembly programmers and compiler makers.” Technical University of Denmark, <https://www.agner.org/optimize/microarchitecture.pdf>, 2025.
- [20] H. Suzuki. “Optimization Notes: Apple M1.” https://github.com/ocxtal/insn_bench_aarch64, 2021.
- [21] C. Lam. “Popping the Hood on Golden Cove.” Chips and Cheese, <https://chipsandcheese.com/p/popping-the-hood-on-golden-cove>, 2021.
- [22] Apple Inc. “Mac Studio Technical Specifications.” <https://www.apple.com/mac-studio/specs/>, 2025.
- [23] gRPC Authors. “gRPC over HTTP2.” <https://github.com/grpc/grpc/blob/master/doc/PROTOCOL-HTTP2.md>, 2015.
- [24] gRPC Authors. “Core concepts, architecture and lifecycle.” <https://grpc.io/docs/what-is-grpc/core-concepts/>, 2023.
- [25] gRPC Authors. “Deadlines.” <https://grpc.io/docs/guides/deadlines/>, 2023.
- [26] P. Henry. “gRPC to AWS Lambda: Is it Possible?” *Coinbase Blog*, March 2019. <https://www.coinbase.com/blog/grpc-to-aws-lambda-is-it-possible>
- [27] “Support for calling gRPC endpoints from Cloudflare Workers.” Discussion #4534, cloudflare/workerd GitHub repository, 2025. <https://github.com/cloudflare/workerd/discussions/4534>
- [28] J. Brandhorst. “The state of gRPC in the browser.” *gRPC Blog*, January 2019. <https://grpc.io/blog/state-of-grpc-web/>
- [29] S. Nelson. “Twirp: a sweet new RPC framework for Go.” *Twitch Blog*, January 2018. <https://blog.twitch.tv/en/2018/01/16/twirp-a-sweet-new-rpc-framework-for-go-5f2febbf35f/>
- [30] Google. “Long Running Operations.” *API Design Guide*. https://cloud.google.com/apis/design/design_patterns#long_running_operations, 2017.
- [31] M. Slee, A. Agarwal, and M. Kwiatkowski. “Thrift: Scalable Cross-Language Services Implementation.” Facebook Technical Paper, April 2007. <https://thrift.apache.org/static/files/thrift-20070401.pdf>
- [32] M. Ulrich. “Addressing Cascading Failures.” In *Site Reliability Engineering: How Google Runs Production Systems*, ch. 22, O’Reilly Media, 2016. <https://sre.google/sre-book/addressing-cascading-failures/>
- [33] N. Kim. “How LinkedIn Adopted Protocol Buffers to Reduce Latency by 60%.” *System Design Newsletter*, 2023. <https://newsletter.systemdesign.one/p/protocol-buffers-vs-json>

- [34] C. Wellons. “Hash Function Prospector.” <https://github.com/skeeto/hash-prospector>, 2018.